NOTA RINGKAS ALL TERMS AND CODING

Panduan Ulang Kaji Komprehensif: Nyahkod Semua Akronim, Teori Compiler, Fungsi Kod & Analogi Mudah

Dokumen Rujukan: Pembentangan Penilai / Pensyarah
Track 1: Agentic Cyber Defence • hackai.my
Format: A4 Master Study Reference

KAMUS AKRONIM A–Z | BEDAH SIASAT SETIAP FUNCTION | ANALOGI ORANG AWAM | TEORI POKOK TERBALIK | SKRIP PERTUTURAN SANTAI

BAHAGIAN 1: GAMBARAN BESAR (APA PROJEK KITA SEBENARNYA?)

🩺 Analogi: "Doktor Pakar Imbasan X-Ray untuk Kod Komputer"

Bayangkan semua sistem perbankan, kata laluan kerajaan, dan laman web di dunia hari ini dikunci menggunakan **"mangga kunci lama"** bernama **RSA** dan **ECC**.

- **Ancaman Masa Depan:** Komputer Kuantum (*Quantum Computer*) bakal tiba. Komputer ini mempunyai formula matematik bernama **Shor's Algorithm** yang boleh memecahkan mangga kunci lama ini dalam beberapa saat sahaja!
- **Serangan HNDL (Curi Dulu, Pecah Nanti):** Penjenayah siber sedang memuat turun data rahsia hari ini, simpan dalam storan mereka, dan tunggu komputer kuantum siap untuk membukanya.
- **Tugas Projek Kita:** Sebelum diserang, perisian kita mengimbas kod sumber (*source code*) perisian lama, mengesan bahagian mana yang masih menggunakan mangga kunci purba, dan mencadangkan gantian mangga kebal kuantum terkini dunia (**NIST PQC FIPS 203 & 204**) — dengan kawalan mutlak manusia (*Zero-Trust Human-in-the-Loop*).

BAHAGIAN 2: KAMUS LENGKAP AKRONIM & SINGKATAN (BAHAGIAN 1)

Setiap kali anda membentang, pensyarah suka menguji sama ada anda tahu maksud singkatan yang anda sebut. Jadual ini menyenaraikan maksud penuh dan huraian bahasa mudahnya:

Akronim	Maksud Penuh *(Full Term)*	Penerangan Bahasa Mudah (Orang Awam Boleh Faham)
PQC	*Post-Quantum Cryptography*	**Kriptografi Kebal Kuantum.** Kaedah kunci keselamatan baharu yang direka khas supaya komputer kuantum pun tidak mampu memecahkannya.
AST	*Abstract Syntax Tree*	**Pokok Struktur Tatabahasa Kod.** Susunan kod dalam bentuk pokok (Pangkal ➔ Dahan ➔ Daun) supaya komputer faham struktur hubungan kod.
SAST	*Static Application Security Testing*	**Ujian Sekuriti Kod Statik.** Mengimbas dan mencari kelemahan dalam baris teks kod *tanpa perlu menjalankan (run)* fail perisian tersebut.
CPU	*Central Processing Unit*	**Otak Komputer.** Cip pemproses elektronik utama dalam komputer yang hanya faham nombor perduaan `0` dan `1` (Bahasa Mesin).
PEG Parser	*Parsing Expression Grammar Parser*	**Enjin Pembaca Tatabahasa Python.** Enjin moden dalam CPython (sejak Python 3.9+) yang membaca kod baris-demi-baris dan membina pokok AST dengan sangat tepat.
JCA	*Java Cryptography Architecture*	**Rangka Kerja Keselamatan Java.** Format rasmi bahasa Java untuk memanggil fungsi sekuriti (contohnya: `KeyPairGenerator.getInstance("RSA")`).
JVM	*Java Virtual Machine*	**Mesin Maya Java.** Enjin perisian yang berat (makan ratusan MB RAM) yang wajib dipasang jika komputer mahu mengkompil atau menjalankan kod Java.
DFA / NFA	*Deterministic / Non-Deterministic Finite Automaton*	**Mesin Logik Pencari Teks.** Model matematik enjin Regex yang membaca huruf satu demi satu dari kiri ke kanan untuk memadankan corak perkataan.
Regex	*Regular Expression*	**Formula Pencari Teks.** Kaedah carian corak rentetan perkataan mendatar seperti fungsi carian teks `Ctrl + F`.
RSA	*Rivest-Shamir-Adleman*	**Kunci Kriptografi Lama (1977).** Algoritma mangga kunci purba berasaskan pendaraban nombor perdana yang bakal dihancurkan oleh komputer kuantum.

NOTA RINGKAS ALL TERMS AND CODING

Sambungan Kamus Akronim & Bedah Siasat Setiap Function Yang Ditaip

KAMUS LENGKAP AKRONIM & SINGKATAN (BAHAGIAN 2: K HINGGA Z)

Akronim	Maksud Penuh *(Full Term)*	Penerangan Bahasa Mudah (Orang Awam Boleh Faham)
ECC / ECDSA	*Elliptic Curve Digital Signature Algorithm*	**Tandatangan Kunci Lengkung.** Digunakan dalam Bitcoin dan kad pintar. Saiz kuncinya kecil, tetapi tetap boleh dipecahkan oleh komputer kuantum.
HNDL	*Harvest Now, Decrypt Later*	**Curi Dulu, Buka Nanti.** Strategi intipan di mana musuh mencuri data internet hari ini, dan simpan sehingga komputer kuantum siap dibina untuk membukanya.
KEM	*Key Encapsulation Mechanism*	**Sampul Kunci Rahsia.** Kaedah membungkus kunci rahsia seperti surat dalam peti keluli sebelum dihantar melalui kabel internet yang tidak selamat.
ML-KEM	*Module-Lattice Key Encapsulation (FIPS 203)*	**Piawaian Mangga Kunci Baharu Dunia.** Nama asalnya *Kyber*. Standard rasmi NIST untuk enkripsi data secara kebal kuantum.
ML-DSA	*Module-Lattice Digital Signature (FIPS 204)*	**Piawaian Tandatangan Baharu Dunia.** Nama asalnya *Dilithium*. Standard rasmi NIST untuk menandatangani dokumen digital secara kebal kuantum.
MTU	*Maximum Transmission Unit*	**Had Muatan Maksimum Paket Internet.** Had berat satu kotak data yang boleh lalu dalam kabel rangkaian internet (standardnya tepat **1,500 bait**).
IND-CCA2	*Indistinguishability under Adaptive Chosen Ciphertext Attack*	**Gred Kebal Paling Tinggi.** Jaminan matematik bahawa walaupun penjenayah mencuba berjuta kali membedah kod rahsia, mereka tiada peluang meneka.
HITL	*Human-in-the-Loop*	**Manusia Memegang Kuasa Veto.** Sistem sekuriti di mana AI dilarang mengubah kod sesuka hati; ia wajib berhenti dan menunggu tandatangan manusia.
RBAC	*Role-Based Access Control*	**Akses Mengikut Pangkat.** Pembahagian kuasa: Pegawai semak kod hanya boleh membaca & meluluskan, Pegawai Admin boleh melihat status sistem.
RAM	*Random Access Memory*	**Memori Ingatan Sementara.** Tempat sistem kita menggantung dan membekukan proses AI semasa menunggu manusia membuat keputusan.
SHA-256	*Secure Hash Algorithm (256-bit)*	**Cap Jari Digital Unik.** Formula matematik yang menukar fail kepada rentetan 64 aksara. Mengubah satu titik pun akan merosakkan seluruh cap jari tersebut.
NIST / FIPS	*National Institute of Standards and Technology*	**Badan Piawaian Amerika Syarikat.** Institut yang meluluskan piawaian keselamatan rasmi dunia (*Federal Information Processing Standards*).
CoT	*Chain-of-Thought*	**Pemikiran Berturutan AI.** Teknik di mana model AI menaakul langkah demi langkah secara logik, bukan meneka secara semberono.

BAHAGIAN 3: BEDAH SETIAP ISTILAH KOD YANG KITA TAIP (DENGAN ANALOGI)

Berikut adalah penjelasan mendalam tentang fungsi sebenar yang kita taip dalam fail kod projek (`pqc_agent/tools.py`, `workflow.py`, dan `audit.py`):

KOD 01 `ast.parse(source_code, filename=file_name)`

Apa Dia: Fungsi bawaan Python yang membaca fail teks kod manusia dan menukarkannya menjadi "objek pokok di dalam memori".
Kenapa Kita Taip: Ini adalah Fasa 2 (*Syntax Analysis*) compiler CPython. Ia menyemak tatabahasa kod dan jika ada kod rosak, ia terus keluarkan `SyntaxError`.

> 🚚 **Analogi LEGO:** Bayangkan orang bawa sebuah kereta mainan LEGO. `ast.parse` ialah mesin pengimbas yang melukis cetak biru (*blueprint*) bahagian mana tayar, enjin, dan pintu kereta tersebut.

KOD 02 `class _PythonCryptoVisitor(ast.NodeVisitor)`

Apa Dia: Kelas khas yang kita cipta dengan mewarisi kelas `ast.NodeVisitor` bawaan Python. Ini menggunakan konsep reka bentuk **Visitor Pattern**.
Kenapa Kita Taip: Membolehkan kita menyelinap dan memintas nod tertentu dalam pokok tanpa perlu menulis kod pencarian manual yang panjang.

> 🥼 **Analogi Inspektor Hospital:** Pokok kod ibarat sebuah hospital besar yang mempunyai beratus-ratus bilik. `_PythonCryptoVisitor` ialah seorang Inspektor Kesihatan yang melawat bilik-bilik itu untuk mencari sama ada ubat tamat tempoh (*kriptografi lama*) digunakan.

NOTA RINGKAS ALL TERMS AND CODING

Sambungan Bedah Siasat Kod: Alias Tracking, Interception, Brek interrupt() & Lejar

tools.py • workflow.py • audit.py
Penjelasan Setiap Baris Yang Ditaip

KOD 03 `visit_Import(self, node) & visit_ImportFrom(self, node)`

Apa Dia: Pintu perangkap automatik (*hook method*). Bila pengimbas ternampak baris `import ...` atau `from ... import ...`, fungsi ini akan menyala dan dipanggil secara automatik.
Kenapa Kita Taip: Untuk merekodkan modul kriptografi lama yang dimasukkan ke dalam fail perisian.

> 🚪 **Analogi Pengimbas Kastam:** Seperti mesin pengimbas di pintu masuk sempadan. Bila ada orang bawa barang larangan masuk, loceng amaran terus berbunyi.

KOD 04 `self._import_aliases[local_name] = full_module`

Apa Dia: Buku catatan nama samaran (*alias lookup table*). Contohnya jika pengaturcara taip: `from Crypto.PublicKey import RSA as kunci_rahsia`.
Kenapa Kita Taip: Pengaturcara sering menukar nama modul. Scanner biasa akan tertipu kerana mencari perkataan "RSA". Tetapi kod kita mencatat bahawa `kunci_rahsia` itu sebenarnya adalah `RSA`!

> 🦹 **Analogi Nama Samaran Penjenayah:** Walaupun penjenayah bernama "Kassim" menukar nama kembarnya kepada "Mat Sabu", detektif polis telah mencatat nama sebenarnya dalam buku rekod. Bila "Mat Sabu" buat jenayah, polis tetap tahu dia adalah Kassim.

KOD 05 `self.generic_visit(node)` — *(SANGAT KRITIKAL!)*

Apa Dia: Perintah khas di hujung fungsi visitor yang menyuruh pelawat: *"Sila teruskan melawat nod-nod anak di bahagian dalam!"*
Kenapa Wajib Ditaip: Jika kita tertinggal baris ini, pelawat kita akan berhenti di pintu depan sahaja dan gagal mengimbas kod yang terletak di dalam fungsi, gelung (*loops*), atau blok *if-else*!

> 🔍 **Analogi Pengawal Keselamatan:** Jika pengawal memeriksa pintu pagar utama tetapi lupa memeriksa bilik tidur dan dapur di dalam rumah, pencuri di dalam bilik tidur tidak akan dijumpai.

KOD 06 `visit_Call(self, node: ast.Call)` — *(TEMPAT TANGKAP KERENTANAN!)*

Apa Dia: Pintu perangkap yang dipanggil apabila kod membuat panggilan fungsi sebenar: `rsa.generate_private_key(key_size=2048)`.
Bagaimana Ia Menangkap RSA:

- Menyemak fungsi: `node.func` (Attribute) ➔ Reconstruct jadi string `rsa.generate_private_key`.
- Menyemak parameter: `node.keywords` ➔ Mencari `key_size` dan membaca nilainya dari `ast.Constant(value=2048)`.
- Bila sah fungsi RSA bersaiz 2048-bit, ia menjana rekod kerentanan berkeyakinan tinggi (**Confidence: 0.85**).

> 🎯 **Analogi Perangkap Tikus:** Tikus boleh lalu merata-rata, tetapi bila ia menyentuh umpan makanan (*fungsi pembuat kunci*), perangkap terus tertutup dan menangkapnya.

KOD 07 `ast.unparse(node)`

Apa Dia: Fungsi penterjemah songsang. Objek nod AST adalah format dalaman komputer yang rumit. `ast.unparse(node)` menukarkannya kembali menjadi teks kod Python asal (cth: `"rsa.generate_private_key"`).
Kenapa Kita Taip: Supaya kita boleh memadankannya dengan mudah terhadap pangkalan data algoritma lama (`LEGACY_CRYPTO_DB`).

> 🗣️ **Analogi Jurubahasa:** Menterjemah semula kod isyarat tangan komputer kembali kepada perkataan bahasa Inggeris yang mudah dibaca oleh manusia.

KOD 08 `interrupt(...) & Command(resume={"decision": "approved"})`

Apa Dia: Tombol brek kecemasan fizikal daripada pustaka LangGraph dalam `node_human_approval()`.
Bagaimana Ia Berfungsi:

- `interrupt()`: Membekukan pelaksanaan ejen AI di dalam memori komputer (RAM / `MemorySaver`). AI dilarang sama sekali menulis kod ke dalam fail sistem.
- `Command(resume=...)`: Kunci penyahbekuan yang hanya dihantar apabila pegawai manusia menekan butang **"Approve"** di Cyber SOC Dashboard selepas menyemak Monaco Diff.

> 🔴 **Analogi Palang Kereta Api:** Kereta api (*Ejen AI*) wajib berhenti di lampu merah. Palang keselamatan tidak akan terbuka secara automatik sehinggalah Renjer Stesen (*Manusia*) memeriksa trek dan menekan suis hijau.

KOD 09 `_compute_chain_hash(entries) & json.dumps(..., sort_keys=True)`

Apa Dia: Formula lejar rantaian cap jari SHA-256 yang menyusun data secara kanonikal (*canonical*).
Formula Matematik: $\text{Hash}_n = \text{SHA256}(\text{CanonicalJSON}(\text{Entry}_n) + \text{Hash}_{n-1})$
Kesan Runtuhan Salji (*Avalanche Effect*): Bermula dari Genesis Hash `"0"*64`. Jika mana-mana staf syarikat cuba mengedit rekod lama, perbezaan satu aksara akan menyebabkan seluruh nombor hash di bawahnya rosak dan mencetuskan amaran **"TAMPER DETECTED"**. `sort_keys=True` memastikan susunan huruf kunci sentiasa seragam mengikut A-Z.

> ⛓️ **Analogi Buku Cek Bertali Dawai Besi:** Setiap halaman diikat dengan mangga bernombor yang bergantung pada nombor mangga halaman sebelum ini. Jika anda koyak satu muka surat di tengah, seluruh tali dawai buku itu terputus.

BAHAGIAN 4: KENAPA POKOK AST INI DIPANGGIL "POKOK TERBALIK"?

🌳 Pokok Salasilah Kod Terbalik (Inverted Tree)

Dalam sains komputer, pokok struktur tidak tumbuh dari tanah ke atas, sebaliknya ia tumbuh dari langit ke bawah. Bayangkan pokok salasilah keluarga di mana moyang paling tua berada di bahagian atas sekali, dan cicit-cicit berada di bawah:

```
[1. PANGKAL POKOK / ROOT (Paling Atas)]: ast.Module
      └── Mewakili keseluruhan fail kod anda.
      |
      └── [2. BATANG POKOK / BRANCH]: ast.FunctionDef ('generate_rsa_keypair')
            └── Menunjukkan kawasan fungsi perisian bermula.
            |
            └── [3. DAHAN POKOK / BRANCH]: ast.Assign (private_key = ...)
                  └── Arahan meletakkan sesuatu ke dalam pembolehubah.
                  |
                  ├── [4A. DAUN KIRI / LEAF]: ast.Name ('private_key')
                  |     └── Nama pembolehubah di sebelah kiri.
                  |
                  └── [4B. DAHAN KANAN / INTERCEPTION NODE]: 🎯 ast.Call
                        └── Enjin _PythonCryptoVisitor kita memintas di sini!
                        |
                        ├── [5A. RANTING / ATTRIBUTE]: rsa.generate_private_key
                        |     └── Diterjemah kembali guna ast.unparse(node).
                        |
                        └── [5B. DAUN PALING HUJUNG / LEAF]: ast.Constant (2048)
                              └── Saiz kunci RSA 2048-bit dikesan & dibongkar!
```

BAHAGIAN 5: PENEMUAN UNIK KITA: APA ITU "PQC MTU PACKET BLOAT"?

🚚 Analogi Lori Pos & Had Lebuhraya Internet (1,500 Bait)

Ini adalah penemuan paling unik projek kita yang membezakannya daripada projek pelajar lain. Terangkan kepada pensyarah menggunakan **Analogi Lori Pos**:

1. Had Lori Pos (Ethernet MTU 1,500 Bait):
Di lebuhraya internet, setiap kabel dan router mempunyai had muatan fizikal yang dipanggil **MTU (Maximum Transmission Unit)** iaitu tepat **1,500 bait**. Bayangkan lori pos yang hanya boleh muatkan barang seberat 1,500 kg.

2. Zaman Kunci Lama (RSA – 256 Bait):
Tandatangan digital RSA lama saiznya hanya **256 bait** (256 kg). Saiznya kecil, jadi satu lori pos boleh muat dengan sangat mudah dan pantas.

Jenis Algoritma	Saiz Tandatangan / Kunci	Kesan Terhadap Had Lori Pos (1,500 Bait MTU)
Legacy RSA-2048	256 Bait	MUAT DALAM 1 LORI (HANYA GUNA 17% RUANG)
NIST ML-KEM-768	1,184 Bait (Kunci Awam)	AMARAN: GUNA 79% MUATAN LORI
NIST ML-DSA-65	**3,309 Bait** (Tandatangan)	💥 220% MELEBIHI HAD LORI! (LORI TERBELAH 3)

Apa Bahayanya Kepada Sistem Syarikat? (*The Availability Threat*):
Kerana saiz kunci kebal kuantum ML-DSA adalah 3,309 bait, sistem rangkaian terpaksa memotong data tersebut menjadi **3 paket berasingan (IP Packet Fragmentation)**. Banyak firewall bank secara automatik akan membuang (*drop*) paket yang terpotong-potong ini kerana mengesyaki ia serangan penggodam. Akibatnya: Laman web menjadi *loading* tanpa henti, sambungan terputus (*timeout*), dan perkhidmatan tergendala! Ejen kita adalah yang pertama dalam industri yang memberi amaran tentang isu ini!

BAHAGIAN 6: CONTOH CARA ANDA CAKAP KEPADA PENSYARAH (GAYA SANTAI TAPI PADAT)

Gunakan gaya pertuturan ini jika pensyarah meminta anda menerangkan keseluruhan sistem dalam 2 minit secara lisan:

"Dr., secara ringkasnya projek kami ini membina satu sistem pengimbas pintar berkonsepkan **Neuro-Symbolic**.

Bahagian **Symbolic** ialah kami meminjam otak pengkompil Python iaitu **CPython PEG Parser**. Melalui modul `ast`, kami menukarkan kod kepada struktur pokok hierarki. Kami cipta satu kelas pelawat bernama `_PythonCryptoVisitor` yang menggunakan **Visitor Pattern**. Kelas ini akan melawat nod `visit_Call`, membongkar fungsi itu menggunakan `ast.unparse`, dan membaca daun pokoknya untuk memeriksa sama ada saiz kunci itu 2048-bit atau kriptografi purba yang boleh dipecahkan oleh komputer kuantum.

Kalau pengaturcara cuba menukar nama pustaka menggunakan 'alias' seperti `import RSA as kunci_saya`, fungsi `visit_ImportFrom` kami akan menyimpannya ke dalam `self._import_aliases`, jadi sistem kami tidak boleh ditipu. Dan hebatnya AST ini berbanding Regex biasa, jika ada komen seperti `# TODO: buang RSA`, AST akan abaikan 100% kerana komen tidak dimasukkan ke dalam pokok tatabahasa.

Kemudian, bahagian **Neural (AI)** kami akan menjana kod gantian terkini piawaian dunia iaitu **NIST FIPS 203 (ML-KEM)**. Tetapi kami tidak benarkan AI menulis kod sesuka hati. Kami pasang brek fizikal menggunakan fungsi `interrupt()` daripada pustaka LangGraph. Proses akan terhenti sepenuhnya di dalam RAM sehingga pegawai manusia menyemak perbezaan kod merah dan hijau di dashboard Monaco kami dan menekan punat `Command(resume='approved')`.

Akhir sekali, setiap kelulusan manusia itu akan dikunci ke dalam rantaian cap jari SHA-256 melalui fungsi `_compute_chain_hash`. Jika ada mana-mana pihak cuba mengubah log audit tersebut, kesan **Avalanche Effect** akan serta-merta merosakkan rantaian hash dan membongkar cubaan menipu tersebut.

Kami juga menemui isu ketersediaan rangkaian di mana saiz tandatangan baharu ML-DSA mencecah 3,309 bait—melebihi had piawaian **Ethernet MTU 1,500 bait sebanyak 220%**, yang boleh melumpuhkan rangkaian jika tidak dikawal. Itulah intipati lengkap projek kami, Dr."

BAHAGIAN 7: 3 SOALAN CEPUMAS PENSYARAH & JAWAPAN MANTAP

❓ Soalan 1: "Kenapa awak guna AST untuk Python, tapi untuk Java awak guna Regex?"

Jawapan: "Untuk Python, kodnya sangat dinamik (boleh tukar nama import sesuka hati), jadi pokok AST wajib digunakan untuk ketepatan 96.8%. Tetapi untuk Java, standard industri menggunakan rangka kerja JCA di mana semua fungsi pembuat kunci dipanggil secara seragam (contoh: `KeyPairGenerator.getInstance`). Jika kita pasang compiler Java penuh (JVM), ia makan 200MB RAM dan mengambil masa lama. Dengan regex DFA/NFA, kita dapat kelajuan di bawah 20 milisaat tanpa perlu pasang perisian tambahan yang berat."

❓ Soalan 2: "Kalau saya letak perkataan RSA dalam komen, sistem awak detect tak?"

Jawapan: "Tidak sama sekali, Dr.! Kerana compiler CPython akan membuang semua komen semasa fasa membina pokok AST. Pokok AST hanya mengandungi baris arahan yang sebenar, jadi amaran palsu (*false positive*) daripada komen adalah 0%."

❓ Soalan 3: "Bagaimana sistem awak pastikan AI tak ubah kod projek tanpa izin?"

Jawapan: "Kami guna fungsi `interrupt()` dari LangGraph. Pelaksanaan ejen AI dibekukan secara fizikal di dalam RAM. Ejen tiada kuasa menulis ke fail sehinggalah pegawai manusia memeriksa kod di skrin Monaco Diff kami dan klik punat Approve."